

Scratch Detection and Localization System- Technical Documentation.

Introduction

My second-year project was a two-stage system that is able to locate and detect scratches (or any other defects) in images automatically. The processing can be divided into binary classification step to determine whether or not the image contains a problem and semantic segmentation step to determine the location of the scratches.

Stage1: Image Classification by using scratch.

Feature Engineering

In order to ensure that the model was lightweight but powerful, I manually created a set of image qualities and texture aspects in OpenCV and NumPy. Here's what I extracted:

Otsu Score - is a global threshold to inform us of the separation of scratch and the background.

FFT Magnitude Mean the average of high-frequency components which are edges and noise shared with scratches.

Laplacian Variance - edge scale which is traditional; the bigger the value, the more probable there are scratches.

Gradient Energy - is produced by first-order spatial gradients, responsive to a sudden change in intensity.

Michelson Contrast- a global contrast measure, which is $(I_{\max} - I_{\min}) / (I_{\max} + I_{\min})$.

Canny Edge Count - Numbers of edges that the Canny function detects, scratches caused a large number of small edges.

All these items are arranged into a Pandas DataFrame and these items are divided into training and test.

Classifier Model

XGBoost was my choice of classifier. The hyper-parameters were:

objective='binary:logistic'

evalmetric='logloss'

randomstate=42

Training Results

The classifier was also very successful with an accuracy of 98%. A sample of classification report has been presented below:

0	0.96	0.95	0.96	716
1	0.99	0.99	0.99	2909
accuracy			0.98	3625
macro avg	0.98	0.97	0.97	3625
weighted avg	0.98	0.98	0.98	3625

Stage2 Localization of Scratch (Semantic Segmentation)

Dataset Structure

I have developed a custom Dataset class that loads RGB scratch images, and pixel-wise binary masks to the images. Images and masks are brought down to 256x 256 size and mask values are binarised to [0,1].

Transformations

The order in which the transforms are applied is: Resize -> ToTensor -> Normalize (this is the usual PyTorch pipeline).

Architecture Model : DeepLabV3ResNet50.

DeepLabV3 was used, with a ResNet50 encoder, the last-stage classifier modified by replacing it with a single binary channel:

Loss Function: Focal Loss

Since there are scarcely many scratch pixels Focal Loss is used to emphasize the difficult examples. It's defined as :

$$L = -\alpha t(1 - Pt)^{\gamma} \log(Pt)$$

.

Optimization

Model was trained using SGD, learning rate 1e-4 and 10 epochs.

Evaluation Metrics And Result:

Pixel Accuracy

Pixel Accuracy = (number of pixels that were adjusted correctly)/ (number of pixels).

Intersection-over-Union (IoU)

$$\text{IoU} = (|\mathbf{P} \cup \mathbf{T}|) / (|\mathbf{P} \cap \mathbf{T}|)$$

Dice Score

$$\text{Dice} = (|\mathbf{P}| + |\mathbf{T}|) / (2|\mathbf{P} \cap \mathbf{T}|)$$

With these metrics, a complete picture of the quality of the segmentation will be created.

===== Final Metrics After Training =====

Pixel Accuracy: 0.9465

IoU Score: 0.0565

Dice Score: 0.1015

High-Performance Pipeline.

The pipeline is simple as illustrated below:

Run XGBoost on a new image.

In case of 1 (bad) prediction, then run the segmentation model.

If 0 (good) - skip segmentation.

Pseudocode:

```
result = model.predict(Xtest)
```

```
if result == 0:
```

```
    model = torch.load('deeplabv3full.pth')
```

```
    output = model(inputimage)
```

```
    predmask = sigmoid(output) > 0.5
```

```
    visualize(image, predmask)
```

```
else:
```

```
    print('Good Image with no need for Localization')
```

Summary of Contributions

Invented a scratch detector that is two-phase.

Fast classification of hand-crafted, lightweight, Applied CV capabilities.

Achieved 98 percent on the classifier.

As a segmentation model, a DeepLabV3ResNet50 was trained.

Focal Loss was used to solve class incongruity.

Pixel Accuracy, Dice and IoU which are reported.

Developed a system of inference which sequences the classification to segmentation.

Developed a mask visualization software.

Conclusion

The present project proves that an automated scratch detector and localization pipeline could be both powerful and effective. Our performance is strong, we have incurred low inference cost, and our segmentation model can still be interpreted due to the blending of the deep learning and traditional CV methods.

The architecture is modular and can be extended to.

Additional defect types

Higher resolution images